



Gruppo SkyNet
skynetunigroup@gmail.com

Proof of Concept degli Agenti

Versione	<i>0.5</i>
Uso	<i>Interno</i>
Destinatari	<i>Team di sviluppo del PoC</i>
Redattori	<i>Samuele Bertin, Marco Barbiero e Sara Ristovic</i>
Verifica	<i>Nome Cognome</i>
Approvazione	<i>Sara Ristovic</i>

Versioni del documento

Ver.	Data	Descrizione	Autore	Verificatore
0.5	2026/08/11	<i>Correzione coerenza con il codice</i>	Samuele Bertin	Nome Cognome
0.4.1	2026/07/26	<i>Aggiunto Qwen come esempio concreto di provider LLM gestito e paragrafo dedicato all'uso dei dati per l'addestramento (RS.5), con l'asimmetria tra Bedrock e API diretta</i>	Marco Barbiero	Nome Cognome
0.4.0	2026/07/26	<i>Allineamento alla progettazione backend: integrazione GitHub read-only con tool-calling, provider LLM via API gestita, aggiunta di modello di dominio, contratto Report, endpoint interni e decisioni architettureali</i>	Marco Barbiero	Nome Cognome
0.3.0	2026/07/24	<i>Inizio ristestura completa del PoC: Backend e infrastruttura</i>	Marco Barbiero	Nome Cognome
0.2.0	2026/07/23	<i>Inizio ristestura completa del PoC: Frontend e tecnologie</i>	Samuele Bertin	Nome Cognome
0.1.0	2026/07/06	<i>Prima stesura: progettazione del PoC degli agenti</i>	Nome Cognome	Nome Cognome

Indice

1	Introduzione e obiettivo	5
2	Perimetro del PoC: cosa implemento e cosa no	5
2.1	Cosa implemento	5
2.2	Cosa non implemento (in questa fase)	6
3	Vincoli, scelte tecnologiche e sostituti di AWS	7
4	Architettura del PoC	9
4.1	Il perimetro in un colpo d'occhio	9
4.2	Un solo scheletro, tre configurazioni	11
4.3	Modello di dominio	13
5	Il contratto di output condiviso	13
6	Integrazione con GitHub	15
6.1	Come l'agente legge dal repository	15
6.2	I quattro controlli su ogni chiamata	16
6.3	Autenticazione degli endpoint interni	16
6.4	Il caso dell'Agente Docs	16
7	Contratto delle API: comunicazione frontend-backend	17
7.1	Due piani di comunicazione distinti	18
7.2	API REST pubbliche (browser → backend)	18
7.3	Canale realtime (browser → backend)	20
7.4	Gli schemi dei DTO principali	20
7.5	API interne (agenti → backend)	21
8	Orchestrazione ed esecuzione	22
8.1	Instradamento deterministico	22
8.2	Ciclo di vita di una Task	22
8.3	Il giro completo end-to-end	23
9	Cosa implemento per ciascun agente	24
9.1	Agente Docs — documentazione inline	25
9.2	Agente Security — scansione vulnerabilità OWASP	25
9.3	Agente Changelog — changelog tecnico	26
9.4	Il filo comune	26
10	Decisioni architetturali (ADR)	26
11	Come validiamo il PoC	27
12	Sintesi e prossimi passi	28

Elenco delle figure

1	Diagramma delle classi	10
2	Architettura a container del PoC	11

3	Modello di dominio persistito su MongoDB, corretto sul codice reale. Le sottoclassi <code>Payload</code> sono state rimosse (ridondanti con la figura 4); <code>AccessLog</code> e <code>ServiceCredential</code> riportano i campi effettivamente presenti negli schemi <code>Mongoose</code>	14
4	Contratto Report	14
5	Facade GitHub read-only	15
6	Agente Docs e destinazione dell'output	17
7	Orchestrazione ed esecuzione	22
8	Ciclo di vita della Task	23
9	Giro completo end-to-end	24
10	Servizio agenti Python	25

1 Introduzione e obiettivo

Code Guardian è un sistema che automatizza audit, manutenzione e documentazione dei repository software. Nella sua forma completa è composto da un orchestratore centrale e da tre agenti intelligenti basati su modelli linguistici: l'agente **Docs** (README e documentazione tecnica), l'agente **Security** (scansione delle vulnerabilità OWASP e verifica delle policy interne) e l'agente **Changelog** (traduzione delle User Stories di uno Sprint in changelog tecnici e di business).

Nella prima stesura di questo documento il PoC era stato ridotto al solo core degli agenti, azionato da un driver a riga di comando che sostituiva sia l'orchestratore sia l'interfaccia utente. Un confronto con il professor Cardin ha messo in luce che questa riduzione lasciava fuori dal perimetro un rischio tecnico reale: non basta dimostrare che un agente basato su LLM legge materiale di progetto reale e produce un output strutturato, accurato e verificabile; occorre anche dimostrare che l'intero stack scelto per il prodotto si integri correttamente, perché diverse di queste tecnologie sono nuove per il team o comunque non ancora validate insieme.

A valle di quella revisione, il proponente ha inoltre respinto la seconda stesura del PoC perché non dimostrava a sufficienza l'integrazione tra le tecnologie scelte: la soglia richiesta è che **almeno l'80% delle tecnologie sia attraversato da un flusso end-to-end reale**, non semplicemente presente nel codice. Il criterio operativo che questo documento adotta è quindi netto: una tecnologia conta come integrata solo se un flusso osservabile e ripetibile la attraversa.

Per questo motivo il perimetro del PoC viene ampliato in due direzioni rispetto alla prima stesura. La prima: non si valida più il solo core dell'agente, ma una fetta verticale end-to-end, dall'interfaccia grafica fino alla persistenza, passando per il backend e per l'agente. La seconda, decisa in fase di progettazione backend e recepita in questa versione del documento: **l'agente dialoga effettivamente con GitHub**, leggendo codice e issue dal vivo attraverso il backend, invece di operare su un clone locale o su fixture. È proprio questa la giunzione che le stesure precedenti non dimostravano.

Restano fuori dal perimetro le parti rimandate a una fase successiva: l'orchestratore centralizzato completo, l'autenticazione, il deploy in produzione su AWS e l'apertura automatica di Pull Request. Questo documento aggiorna quindi perimetro, architettura e scelte tecnologiche del PoC, riflettendo lo stack realmente adottato e le decisioni di progettazione prese a valle della prima stesura.

2 Perimetro del PoC: cosa implemento e cosa no

Questa è la sezione più importante da condividere con il team, perché definisce le aspettative. Il PoC copre una fetta verticale per ciascuno dei tre agenti: una singola operazione rappresentativa, portata dall'inizio alla fine, sopra un'infrastruttura condivisa. Non copre gli agenti nella loro interezza né i casi d'uso di contorno.

2.1 Cosa implemento

- Un grafo di agente eseguibile (basato su LangGraph, in Python), con i suoi nodi, la gestione del timeout e la gestione strutturata degli errori.
- Un'astrazione del provider LLM (**LLMProvider**) con implementazione attiva verso un **API cloud gestita** (ad esempio Qwen) raggiunta via HTTPS con chiave di servizio, e un'implementazione per AWS Bedrock scritta ma non attivabile in questa fase (credenziali assenti).
- L'integrazione con **GitHub in sola lettura**: l'agente legge codice e issue del repository selezionato attraverso una *facade* esposta dal backend, che l'agente utilizza come insieme di *tool* LangGraph (§ 6).

- Un filtro sui linguaggi supportati (TypeScript, JavaScript, Python) e un limite dimensionale dell'ambito, applicati alla costruzione del contesto d'analisi.
- Una fetta per ciascun agente: documentazione inline per Docs, scansione vulnerabilità per Security, changelog tecnico per Changelog.
- Un contratto di output comune (l'oggetto **Report**), persistito su MongoDB e reso via API REST (§ 5).
- Un backend REST (NestJS su Node.js/TypeScript) che espone gli endpoint per avviare un agente, recuperarne lo stato e leggerne il report, che custodisce il token GitHub cifrato e che espone agli agenti la facade di lettura.
- Un'interfaccia grafica (React, con Vite, TanStack Router, pnpm) che consente di scegliere repository, ambito e operazione, avviare l'esecuzione e visualizzare il report.
- Un ambiente containerizzato (Docker Compose) che fa girare frontend, backend, servizio agenti, database e servizi di appoggio in locale.
- L'isolamento dei prompt in file esterni e i test con provider LLM simulato (per copertura e ripetibilità).

2.2 Cosa non implemento (in questa fase)

- L'orchestratore deterministico completo. Il PoC ne include la forma minima necessaria a instradare un'operazione verso il suo agente (§ 8), ma non la gestione di batch complessi né la dashboard di monitoraggio.
- La scrittura su GitHub: nessuna Pull Request viene aperta. L'agente si ferma alla proposta (un diff), coerentemente con il vincolo RS.3 che vieta agli agenti autonomi la scrittura sul repository.
- L'infrastruttura AWS (hosting, Bedrock, SES, S3): sostituita da esecuzione locale containerizzata e da un'API LLM gestita (§ 3). Il modello **non** viene eseguito localmente.
- Registrazione, login, configurazione credenziali complete, notifiche email, dashboard di monitoraggio: casi d'uso di contorno. Il PoC assume un token GitHub già configurato via variabile d'ambiente o schermata minimale.
- Le operazioni degli agenti non scelte come fetta: README e documentazione API per Docs, changelog di business per Changelog, e la seconda funzione dell'agente Security (policy-as-code su **POLICY.md**), inclusa come variante opzionale ma non obbligatoria.

Tabella 1: Sintesi del perimetro: cosa entra nel PoC e cosa è rimandato.

Elemento	Nel PoC	Rimandato
Grafo agente + gestione errori/timeout	✓	
Provider LLM via API gestita (adapter Bedrock predisposto)	✓	
Integrazione GitHub read-only (codice + issue, dal vivo)	✓	
Facade GitHub come tool LangGraph	✓	
Contratto Report condiviso, persistito su MongoDB	✓	
Orchestrazione minima (instradamento operazione → agente)	✓	
Docs: documentazione inline	✓	
Security: scansione vulnerabilità OWASP	✓	
Changelog: changelog tecnico da GitHub Issues	✓	
Frontend React (Vite, TanStack, pnpm)	✓	
Backend NestJS + persistenza MongoDB	✓	
Orchestratore deterministico completo		✓
Apertura Pull Request su GitHub		✓
Hosting / Bedrock su AWS		✓
Auth completa, email, dashboard		✓
Docs: README e API docs		✓
Security: policy-as-code su <code>POLICY.md</code>	(opzionale)	
Changelog: changelog di business		✓

3 Vincoli, scelte tecnologiche e sostituti di AWS

Il capitolato prevede AWS in tre punti: gli LLM tramite un servizio cloud (Bedrock), l'hosting e l'infrastruttura, e il database MongoDB. A questi si aggiungono le scelte relative al linguaggio del core degli agenti e all'intero stack di backend e frontend. Poiché in questa fase non sono disponibili le credenziali AWS, ogni servizio cloud è sostituito da un equivalente locale scelto in modo che la migrazione futura cambi la configurazione, non il codice applicativo. Di seguito le motivazioni.

Python. È il linguaggio scelto per il core degli agenti. Oltre ai vantaggi tecnici consolidati (ecosistema maturo per l'integrazione con modelli linguistici tramite LangGraph, LangChain e SDK dei provider, tipizzazione graduale con i type hints, ampia disponibilità di librerie per parsing e testing) la scelta è guidata da un criterio pragmatico: è un linguaggio che il team già conosceva. Con i tempi ristretti del PoC, evitare la curva di apprendimento di un nuovo linguaggio sulla parte più rischiosa del sistema ha permesso di concentrare lo sforzo sulla validazione del rischio tecnico reale.

MongoDB. È il database scelto per la persistenza dei report. È un database documentale (NoSQL) particolarmente adatto qui perché l'oggetto **Report** è naturalmente un documento semi-strutturato, con blocchi di tipo variabile e campi opzionali: rappresentarlo come documento JSON evita il sovraccarico di normalizzazione di uno schema relazionale rigido, mantenendo la validazione della struttura a livello applicativo. È inoltre coerente con l'ecosistema Node.js/TypeScript del backend, per cui esistono driver e ODM (Mongoose) maturi e ben integrati con NestJS.

Infrastruttura: Docker Compose per il PoC, AWS in produzione. Nel PoC l'infrastruttura è containerizzata: frontend, backend, servizio agenti, database e servizi di appoggio girano in container orchestrati localmente con Docker Compose. Scelta transitoria, motivata dalla necessità di iterare rapidamente senza i tempi e i costi di provisioning cloud. La migrazione ad AWS resta l'obiettivo delle fasi successive. La tabella 2 elenca le corrispondenze uno-a-uno tra i servizi locali del PoC e i servizi AWS di destinazione.

Tabella 2: Sostituti locali dei servizi AWS. Nessuna sostituzione tocca il codice applicativo: cambia solo la configurazione.

Target AWS	Sostituto nel PoC	Cosa cambia alla migrazione
ECS / Fargate	Docker Compose	Unità di deploy identica (l'immagine): serve la task definition
Bedrock	API LLM gestita	Si attiva l'adapter <code>BedrockProvider</code> e si cambia <code>LLM_PROVIDER</code>
SES	Mailpit (SMTP)	Cambia l'host SMTP nel transport
S3	MinIO (S3-compatibile)	Cambia l'endpoint del client
DocumentDB	MongoDB container	—
ElastiCache	Redis container	—

Provider LLM: API gestita, non esecuzione locale. Nella prima stesura l'agente era vincolato all'uso di Claude tramite l'API diretta di Anthropic; la seconda stesura aveva ripiegato su un modello open source servito localmente. Entrambe le impostazioni sono state riviste in fase di progettazione, per una ragione precisa: **un modello eseguito localmente non produrrebbe conoscenza utile**. Il PoC deve dirci se il prodotto è realizzabile con le tecnologie di capitolato; un modello ridotto su CPU darebbe latenze, costi e qualità che non somigliano a nulla di ciò che vedremo in produzione, e i prompt tarati su quel modello andrebbero riscritti quando si passa a Bedrock.

Il PoC utilizza quindi un'**API cloud gestita** (ad esempio Qwen, tramite l'endpoint DashScope di Alibaba Cloud Model Studio), raggiunta via HTTPS con chiave di servizio, dietro l'astrazione `LLMProvider`. Il criterio di scelta del fornitore non è arbitrario: si adotta la **stessa famiglia di modelli** che sarà poi servita da Bedrock. Bedrock, infatti, non è un modello ma un canale di distribuzione: serve modelli di terze parti sotto contratto AWS. Usando l'API diretta dello stesso fornitore durante il PoC, il comportamento osservato (prompt, tool-calling, consumo di token) è quello vero, e la migrazione a Bedrock si riduce al trasporto. L'astrazione `LLMProvider` resta il punto di forza: cambiare provider è un cambio di configurazione, non una riscrittura.

L'esempio di Qwen è concreto proprio perché supera questo criterio: i modelli Qwen3 sono già disponibili come modelli gestiti su Amazon Bedrock (Milano inclusa tra le regioni), e la famiglia Qwen3-Coder è orientata all'analisi di codice e al *function calling*, il meccanismo su cui poggia l'intera integrazione GitHub descritta in § 6. Durante il PoC il modello si raggiunge via DashScope in modalità compatibile con l'API OpenAI, il che permette all'adapter di riusare l'SDK OpenAI cambiando solo `base_url` e chiave; alla migrazione si attiva `BedrockProvider` puntando allo stesso modello, senza toccare prompt né grafi. La scelta del taglio specifico (ad esempio Qwen3-Coder nella variante da 30B o 480B) e la sua validazione sul golden set restano da confermare: il requisito di accuratezza $\geq 85\%$ (RQ.5) si misura sul modello effettivamente adottato, non si assume.

Backend: TypeScript, Node.js, NestJS. Il backend è in TypeScript su runtime Node.js, con NestJS. Il framework impone una struttura modulare (moduli, controller, provider/service)

che riduce il rischio di disorganizzazione quando più persone contribuiscono in parallelo; l'introduzione delle dipendenze nativa semplifica anche l'introduzione dei test. TypeScript condiviso tra backend e frontend permette di riusare le stesse definizioni di tipo per il contratto **Report**, riducendo il disallineamento tra le due parti dello stack.

Frontend. L'interfaccia grafica è in React. Alcuni strumenti dell'ecosistema sono stati indicati come fortemente consigliati dall'azienda committente, che offre supporto diretto in caso di difficoltà:

- **Vite:** bundler/dev-server scelto per i tempi di avvio e hot-reload ridotti. Per il **Proof of Concept (PoC)** è stato adottato nell'impostazione standard (*vanilla*) mediante il plugin ufficiale `@vitejs/plugin-react`. L'integrazione con i motori **Oxc** e **Rolldown**, indicati dall'azienda con garanzia di supporto, è prevista per i successivi sviluppi ma non impiegata nella versione attuale del PoC, riducendo così il rischio legato alla loro maturità e documentazione.
- **pnpm:** gestore di pacchetti scelto per la gestione efficiente delle dipendenze (store condiviso e deduplicato, installazioni rapide) e per coerenza con le convenzioni interne dell'azienda.
- **TanStack Router:** router per React scelto per la tipizzazione nativa di rotte e parametri, integrata con TypeScript. Anche questa è una scelta indicata dall'azienda.

4 Architettura del PoC

4.1 Il perimetro in un colpo d'occhio

Il PoC è una web app locale containerizzata. L'utente interagisce con l'interfaccia grafica (React), che chiama il backend (NestJS) tramite API REST e riceve gli aggiornamenti di avanzamento via WebSocket. Il backend custodisce il token GitHub cifrato, avvia l'agente richiesto, ne riceve il report, lo persiste su MongoDB e lo restituisce al frontend. L'agente, al proprio interno, usa il provider LLM (via API gestita) e legge dal repository **non** in autonomia, ma chiedendo al backend attraverso la facade di sola lettura. Questa inversione è il cuore della decisione ADR-1 (§ 10) ed è ciò che tiene il token confinato in un solo servizio.

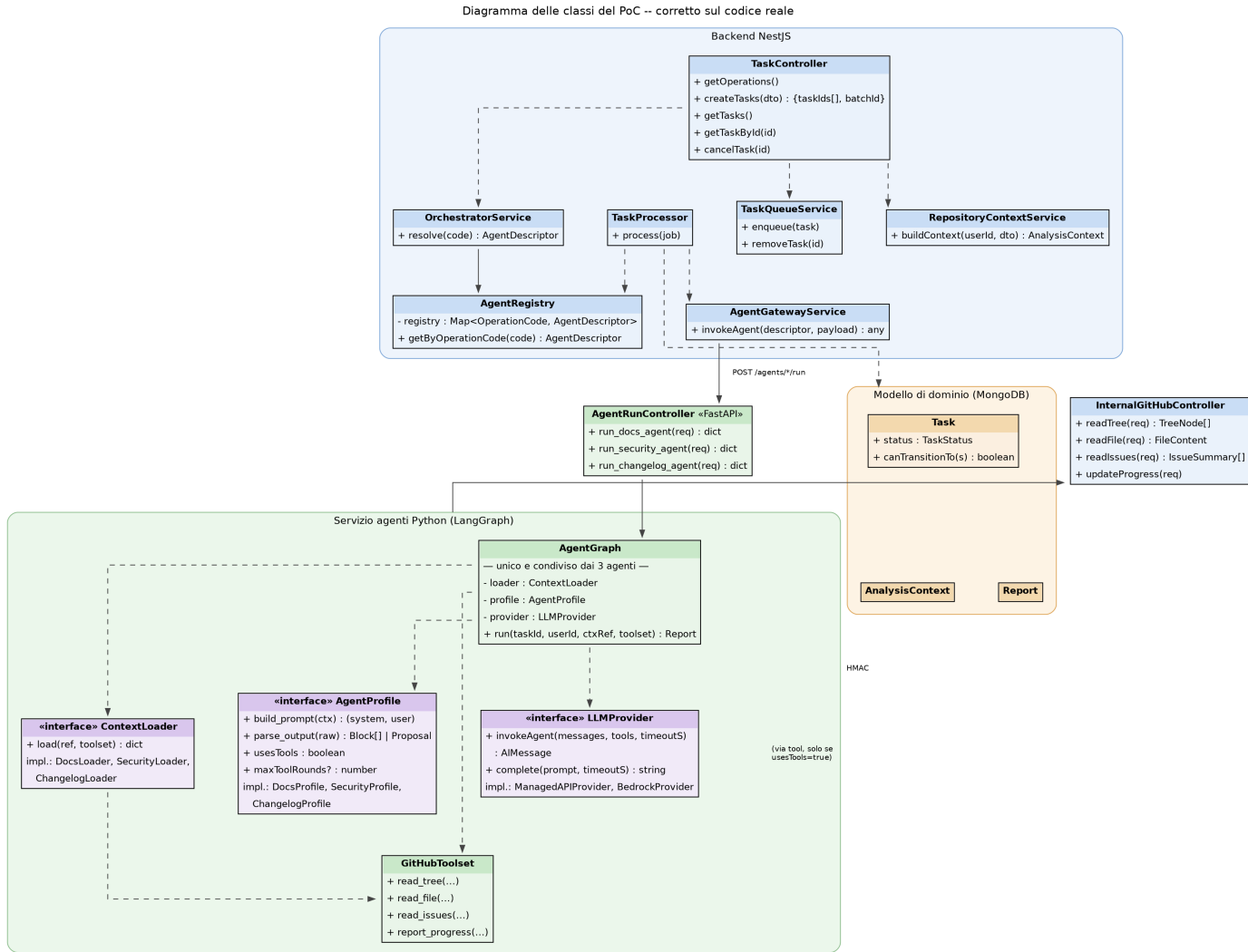


Figura 1: Diagramma delle classi del PoC.

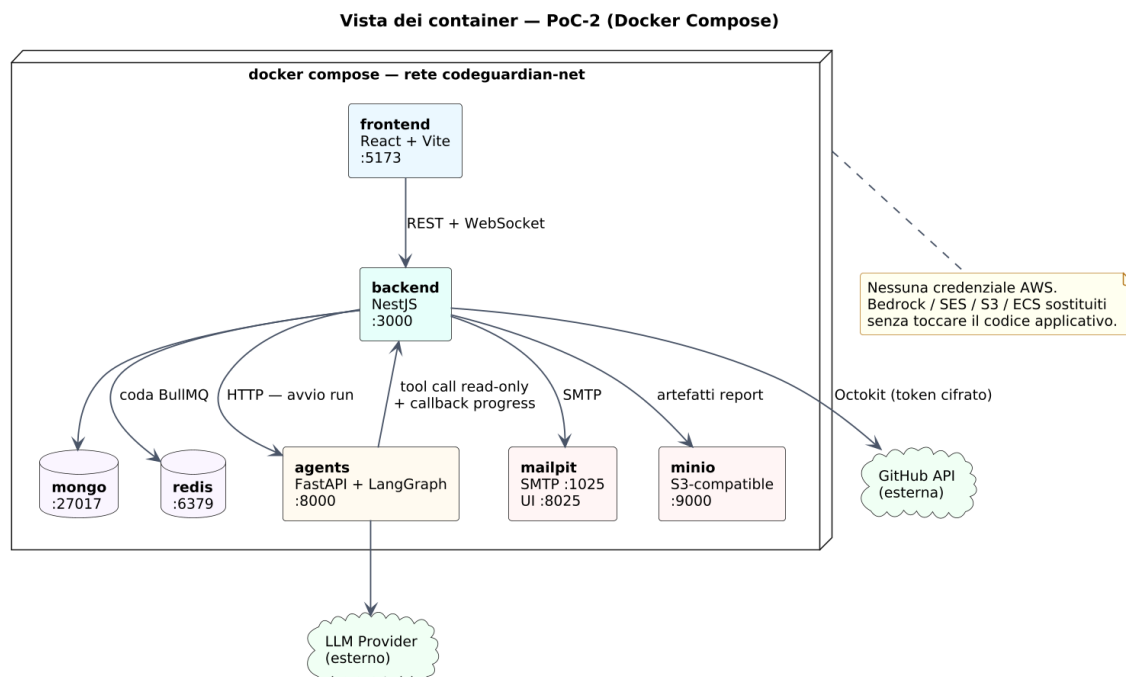


Figura 2: Architettura a container. Tre servizi portano codice nostro (frontend, backend, agents; i restanti sono infrastruttura. Le uniche connessioni verso l'esterno sono backend→GitHub e agents→provider LLM. La freccia agents→backend (traffico inverso) è la facade di lettura descritta in § 6.

4.2 Un solo scheletro, tre configurazioni

L'osservazione che rende economico coprire tutti e tre gli agenti resta valida: i tre agenti condividono la stessa struttura interna. Il grafo LangGraph è fisso, ed è più articolato di un semplice “carica contesto → costruisci prompt → chiama il modello”: oltre ai nodi di caricamento contesto, composizione del prompt, invocazione del modello, validazione/parsing e assemblaggio del report, il grafo include un nodo di **esecuzione dei tool** con un arco condizionale che, quando il modello richiede una chiamata, riporta l'esecuzione al nodo di invocazione del modello: è il loop che permette a un agente di leggere un file, ricevere il contenuto, e decidere se leggerne un altro prima di rispondere. Questo loop è predisposto nello scheletro condiviso ma viene effettivamente percorso solo dagli agenti che lo attivano. Ciò che varia da un agente all'altro resta comunque confinato in due porte (nel senso di *ports-and-adapters*). Lungo il grafo scorre un unico oggetto di stato, **AgentState**, che i nodi leggono e arricchiscono: parte da {**context_ref**, **profilo**}, si popola con il contesto caricato, il prompt, la cronologia dei messaggi scambiati col modello (comprese le eventuali chiamate a tool e i loro risultati), la risposta grezza, i blocchi o la proposta, e infine il report; un campo **error** opzionale devia verso lo stato fallito.

Le porte che variano sono due:

- **ContextLoader** — dato un riferimento, restituisce il contesto iniziale da dare al modello. Nel PoC-2 gli adattatori **non** leggono più da clone locale o fixture: leggono dal repository reale attraverso i tool della facade GitHub. Per Docs e Changelog questo passo carica *tutto* ciò che serve (rispettivamente: albero e contenuto dei file dell’ambito per Docs; le issue chiuse per Changelog), perché questi due agenti non hanno tool a disposizione durante l’esecuzione del grafo e devono ricevere l’intero materiale già pronto nel prompt. Per Security, invece, **ContextLoader** carica solo l’albero del repository (e l’eventuale POLICY.md): il contenuto dei singoli file non viene letto in questo passo, ma più avanti nel grafo, durante il ciclo di invocazione del modello, tramite i tool descritti sotto.

- **AgentProfile** — incapsula le cose specifiche di ogni agente: costruire il prompt (caricando il template esterno e iniettando il contesto: è qui che vive l'isolamento dei prompt) e rileggere la risposta (validarla e trasformarla nei blocchi del report secondo lo schema dell'agente). Il profilo dichiara inoltre se l'agente ha accesso ai tool di lettura durante l'esecuzione (`usesTools`) e, in caso affermativo, quante iterazioni del ciclo modello→tool→modello sono ammesse prima di considerare l'agente fallito (`maxToolRounds`). È questo flag, letto dallo scheletro condiviso, a decidere se il grafo percorre il nodo di esecuzione tool oppure lo salta: per Docs e Changelog è disattivato, per Security è attivo con un tetto di iterazioni.

C'è poi una terza porta, `LLMProvider`, condivisa e identica per tutti: cambia solo per configurazione, non per agente. È questa astrazione a rendere il passaggio a Bedrock un cambio di configurazione. Il metodo principale non è una singola chiamata testo-a-testo, ma un'invocazione che riceve anche l'elenco dei tool disponibili (eventualmente vuoto) e la cronologia dei messaggi, così da poter restituire sia una risposta finale sia una richiesta di chiamata a un tool.

Rispetto alla prima stesura, il core dell'agente non è più invocato da un driver a riga di comando, ma da un use case del backend NestJS, che inietta `context_ref` e `profilo`, avvia il grafo ed espone il `Report` risultante come risposta REST.

```
// Porte (interfacce) -- l'unico codice che varia per agente
interface ContextLoader { load(ref): LoadedContext } // via facade
    GitHub
interface AgentProfile { build_prompt(ctx): Prompt
                        parse_output(raw): Block[] | Proposal
                        usesTools: boolean // false per
                        Docs, Changelog
                        maxToolRounds?: number } // usato solo
                        se usesTools = true

interface LLMProvider { // condiviso
    invokeAgent(messages: Message[], tools: Tool[], timeoutS): AIMessage
    complete(prompt: string, timeoutS): string // scorciatoia
    quando tools = []
}

// Composizione degli adapter per agente
Docs -> GitHubCodeLoader + DocsInlineProfile (usesTools: false)
Security -> GitHubTreeLoader + OwaspScanProfile (usesTools: true,
    maxToolRounds: N)
                                (+ POLICY.md, opzionale)
Changelog -> GitHubIssuesLoader + ChangelogTechProfile (usesTools: false
    )

% Backend NestJS -- sostituisce il driver CLI
POST /contexts { repoOwner, repoName, ref, scopeType, paths? } -> 201 {
    contextId }
GET /operations -> elenco OperationDescriptorDto disponibili per l'UI
POST /tasks { contextId, operations: OperationCode[] } -> 202 { taskIds:
    string[], batchId }
GET /tasks?{query} -> TaskSummaryDto[]
GET /tasks/:id -> TaskDetailDto
POST /tasks/:id/cancel -> void
GET /reports/:id -> Report persistito su MongoDB
GET /reports/:id/export?format=md -> file scaricabile (Report ->
    Markdown)
WS task.progress / task.updated -> avanzamento e stato finale (non
    polling)
```

Listing 1: Porte dell'agente e composizione degli adapter. `ContextLoader` e `AgentProfile` variano

per agente; il ciclo di tool-calling è parte dello scheletro condiviso ma è attivo solo per i profili con `usesTools=true`.

Il frontend costruisce prima l'`AnalysisContext` tramite `POST /contexts` (repository, ambito, riferimento), quindi avvia una o più operazioni con `POST /tasks`, che risponde immediatamente con gli identificativi delle task create in stato `PENDING` e non con il report. L'avanzamento e la transizione di stato (`COMPLETED`, `FAILED`, `CANCELLED`) arrivano via WebSocket (`task.progress`, `task.updated`), non tramite interrogazione ripetuta di `GET /reports/:id`. Gli endpoint `/internal/*` restano riservati alla comunicazione agente→backend e non sono mai chiamati dal frontend.

4.3 Modello di dominio

Il diagramma in figura 3 mostra le entità persistenti su MongoDB. Al centro c'è `Task`: ogni esecuzione richiesta dall'utente ne è un'istanza. Attorno ruotano `AnalysisContext` (su cosa si lavora: repository, riferimento risolto a SHA, ambito), `OperationCode` (cosa si fa: la chiave che l'orchestratore usa per instradare) e `Report` (cosa ne è uscito). L'`AccessLog` registra ogni lettura effettuata su GitHub per conto di una task – nello schema reale i campi sono `taskId`, `endpoint` e `resource` (non esistono un `userId` né un campo `operation` propri del log: l'endpoint richiamato già identifica il tipo di lettura) – ed è l'evidenza che rende verificabile il vincolo di sola lettura.

`AnalysisContext` è deliberatamente persistente e non passato inline: più task dello stesso avvio lo condividono senza duplicarlo, e il pin del riferimento a uno SHA rende i report riproducibili anche dopo nuovi commit sul repository. Il campo opzionale `sprintId` è usato solo dall'operazione Changelog, per filtrare le issue sulla milestone richiesta. `AnalysisContext` non espone alcun metodo di validazione dimensionale: il controllo sulla dimensione del contesto (`max_scope_chars`) è implementato lato agente Python, nel nodo `componi_prompt` del grafo LangGraph.

Il `Report` è un envelope unico e generico: qualunque sia l'agente che lo produce, la struttura è sempre `{..., body: Block[], proposal?: Proposal, error?: ReportError}`.

Infine, `ServiceCredential` custodisce il token GitHub cifrato: oltre a `ciphertext` e `authTag`, lo schema reale include `iv` (vettore di inizializzazione) e `salt`, entrambi necessari alla cifratura AES; il campo `provider` è una stringa libera (di fatto sempre "GITHUB" in questo PoC), non un'enumerazione chiusa. Gli sviluppatori implementano questi schemi con Mongoose.

5 Il contratto di output condiviso

Il nodo finale del grafo, *Assembla report*, produce sempre lo stesso oggetto `Report`, qualunque sia l'agente. È questo a rendere i tre output un contratto unico: sia il backend sia il frontend trattano sempre un `Report`, senza sapere quale agente l'ha prodotto. La struttura è un *envelope*: un'intestazione comune, un corpo fatto di blocchi ordinati, e un eventuale artefatto di proposta (il diff). Il `Report` viene persistito su MongoDB dal backend e reso via API REST al frontend, che lo renderizza nell'interfaccia grafica.

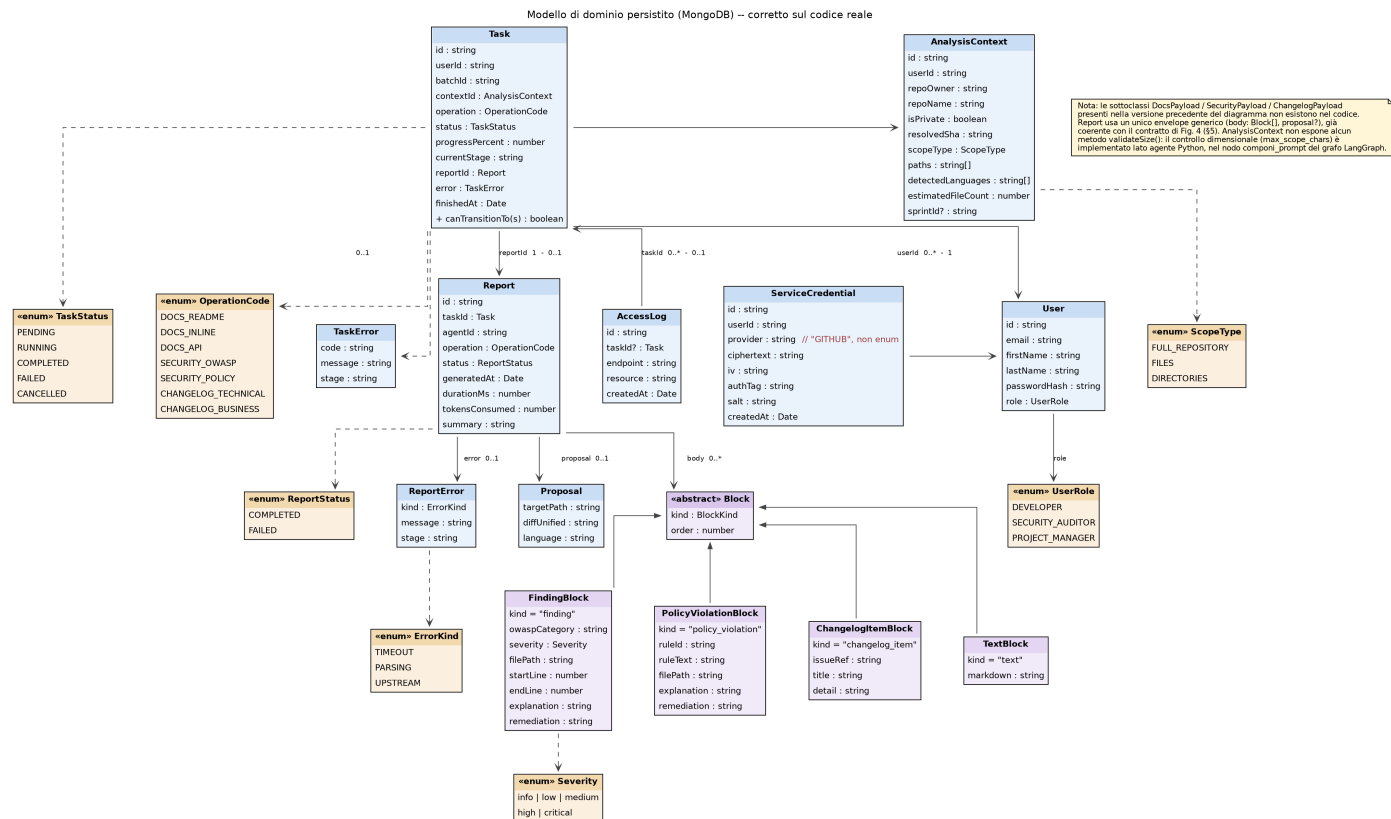


Figura 3: Modello di dominio persistito su MongoDB, corretto sul codice reale. Le sottoclassi Payload sono state rimosse (ridondanti con la figura 4); AccessLog e ServiceCredential riportano i campi effettivamente presenti negli schemi Mongooose.

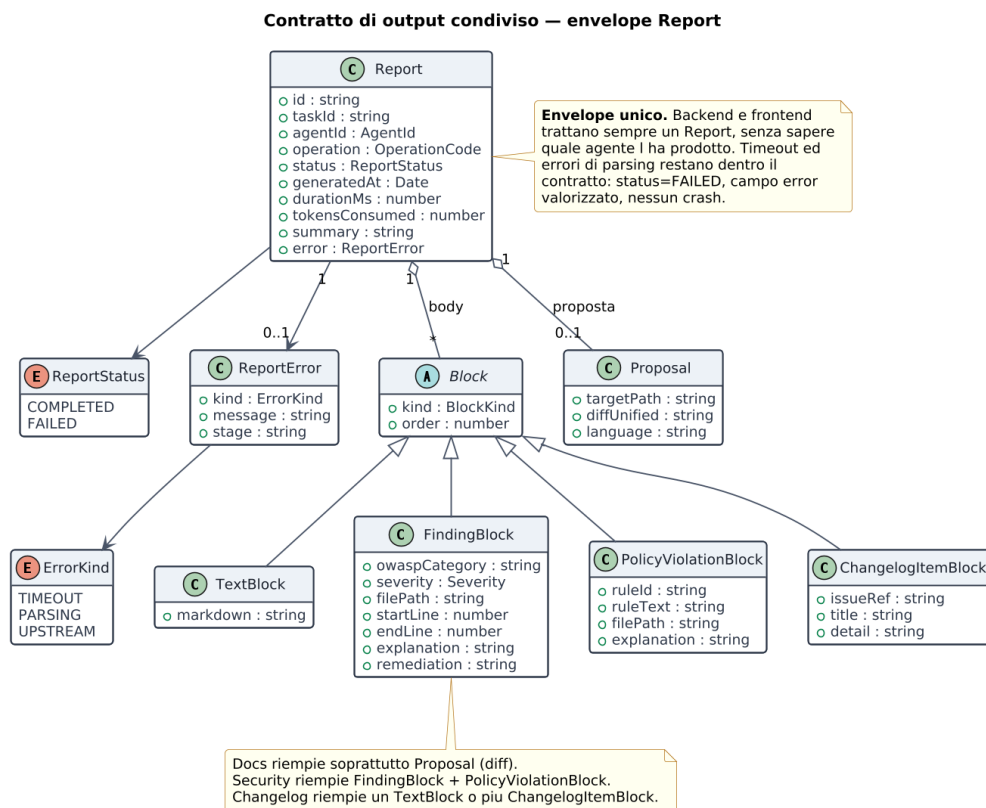


Figura 4: Il contratto Report: envelope comune, blocchi polimorfi nel corpo, proposta opzionale. Timeout ed errori di parsing restano dentro il contratto (status=FAILED, campo error valorizzato), senza provocare crash.

Ogni agente riempie lo stesso envelope in modo diverso: Security con `FindingBlock` ed eventuale `PolicyViolationBlock`; Docs soprattutto con una `Proposal` di diff; Changelog con un `TextBlock` o più `ChangelogItemBlock`. In tutti i casi, timeout ed errori di parsing non provocano crash: valorizzano `error` e portano `status` a `FAILED`, restando dentro il contratto; il frontend mostra questi stati con un componente di errore dedicato, senza gestione speciale lato backend.

6 Integrazione con GitHub

Questa è la sezione che gli sviluppatori devono leggere con più attenzione, perché è dove è più facile sbagliare in un modo che viola un vincolo di sicurezza. La regola da tenere ferma è una sola: **il token GitHub non lascia mai il backend NestJS, e gli agenti non chiamano mai GitHub direttamente.**

6.1 Come l'agente legge dal repository

L'agente non possiede il token e non conosce l'URL di GitHub. Quando ha bisogno di leggere qualcosa, invoca un *tool* LangGraph che, sotto il cofano, chiama un endpoint interno del backend. È il backend a decifrare il token, chiamare GitHub in sola lettura, registrare l'accesso e restituire il contenuto. La figura 5 mostra le classi coinvolte.

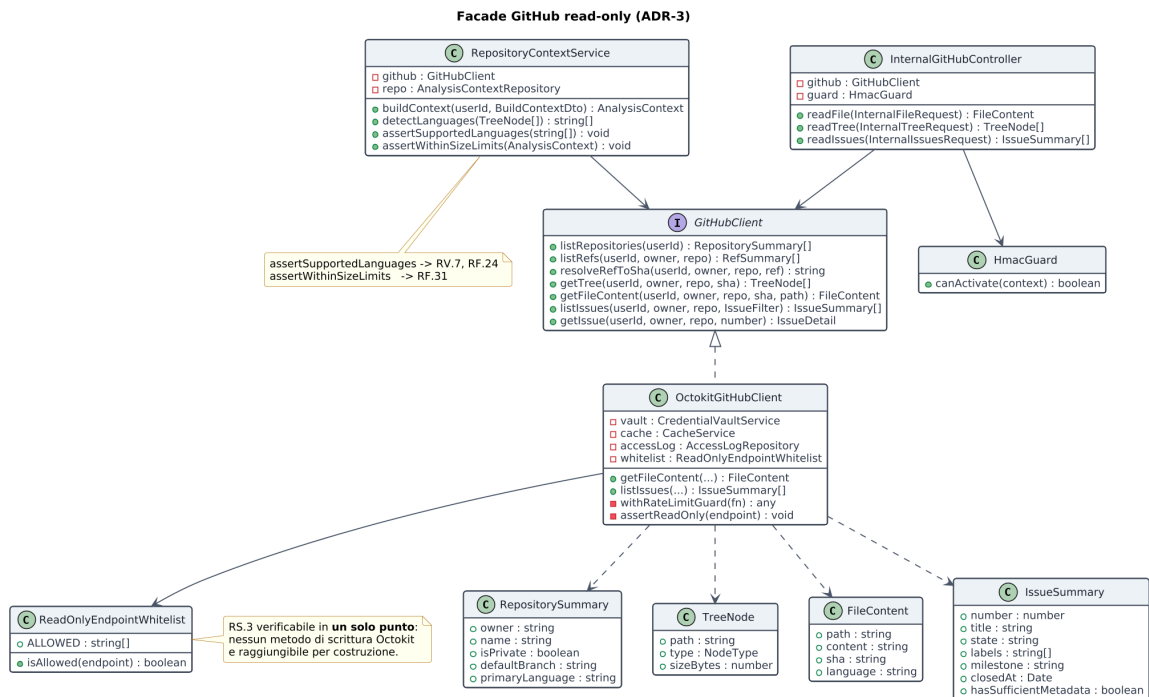


Figura 5: La facade GitHub. `ReadOnlyEndpointWhitelist` rende il vincolo di sola lettura verificabile in un solo punto: nessun metodo di scrittura è raggiungibile per costruzione. Gli endpoint `/internal/*` sono usati dagli agenti e protetti da HMAC.

I tool disponibili all'agente sono tre, tutti di lettura: `read_tree` (struttura del repository a un dato SHA), `read_file` (contenuto di un file) e `read_issues` (elenco e dettaglio delle issue, filtrabili). Mentre l'agente Security sfrutta un vero tool-calling autonomo e iterativo per esplorare i file, gli agenti Docs e Changelog seguono un flusso deterministico: un loader Python pre-carica interamente il contesto (file di codice o issue) prima dell'invocazione del modello, al quale viene esplicitamente vietato l'uso di tool esterni, ed è ciò che le stesure precedenti non dimostravano.

6.2 I quattro controlli su ogni chiamata

Ogni lettura verso GitHub attraversa quattro meccanismi, che gli sviluppatori implementano una sola volta in `OctokitGitHubClient`:

- **Whitelist** (`ReadOnlyEndpointWhitelist`): se un endpoint non è nell'elenco chiuso di operazioni di lettura, la chiamata non parte. Il vincolo RS.3 non dipende dall'attenzione di chi scrive il codice, ma è garantito per costruzione.
- **Cache Redis** con chiave `repo@sha:path`: poiché il contesto è pinnato a uno SHA, il contenuto di un file a quel commit è immutabile e la cache non può servire dati stantii. Serve a contenere il consumo di rate limit quando l'agente rilegge lo stesso file in iterazioni diverse.
- **Rate limit guard**: legge le intestazioni di quota di GitHub e, in prossimità dell'esaurimento, rallenta invece di fallire.
- **AccessLog**: ogni chiamata lascia una riga con task, endpoint e risorsa. È l'evidenza che rende RS.3 dimostrabile invece che dichiarata.

6.3 Autenticazione degli endpoint interni

Gli endpoint `/internal/github/*` e `/internal/tasks/:id/progress` non sono esposti al frontend e non compaiono nello Swagger pubblico. Sono raggiungibili solo dalla rete Docker e autenticati con **HMAC** tramite un segreto condiviso tra backend e servizio agenti (`INTERNAL_SHARED_SECRET`), perché il chiamante non è un utente ma un servizio. Gli sviluppatori del servizio agenti firmano ogni richiesta; gli sviluppatori del backend verificano la firma in un guard dedicato.

6.4 Il caso dell'Agente Docs

L'Agente Docs merita una nota, perché è l'unico che *produce un file che potrebbe già esistere* nel repository, e quindi il vincolo di sola lettura sembra configgere con la sua funzione. Non è così: l'agente legge dal repository ma non ci scrive.

Agente Docs -- dialogo con GitHub e destinazione dell'output (corretto sul codice reale)

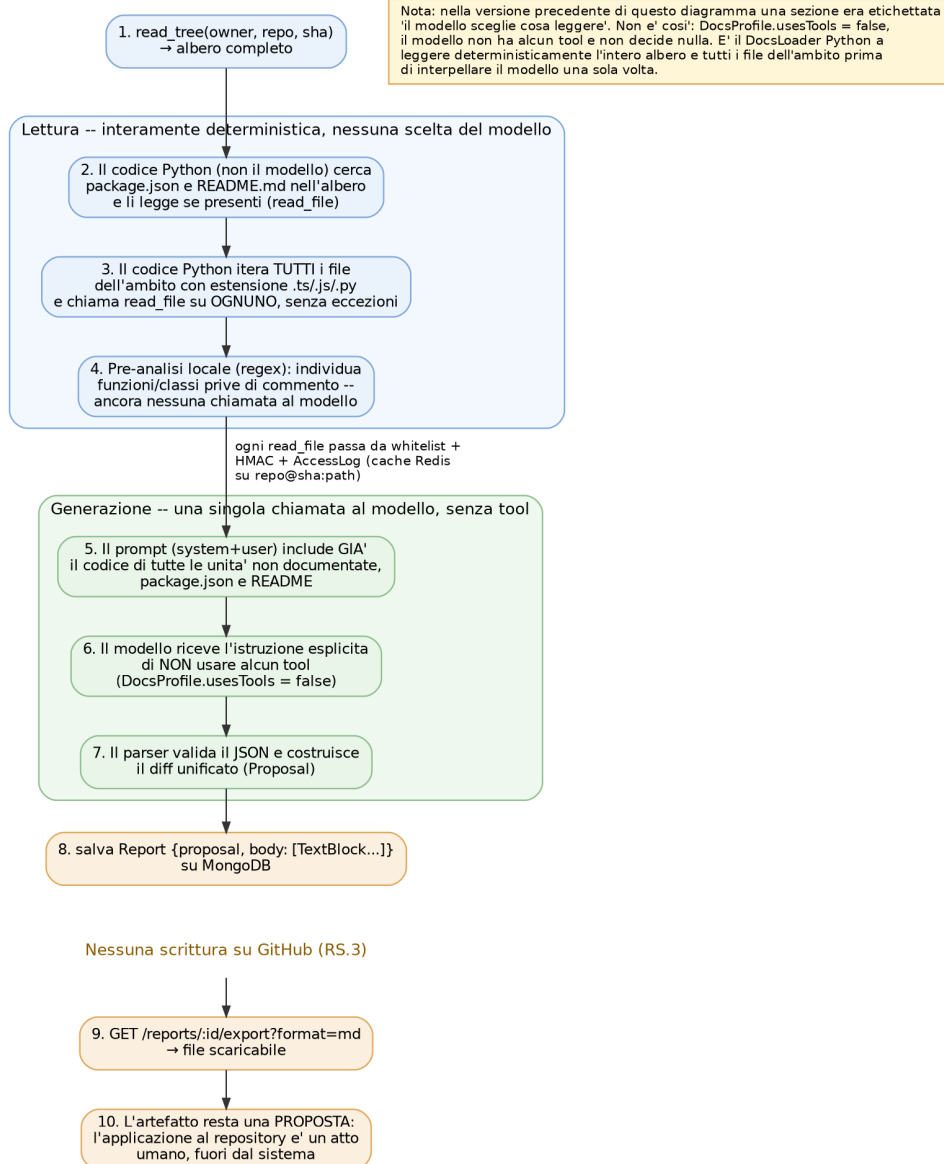


Figura 6: Agente Docs: legge in modo più esteso degli altri (deve prima capire che progetto è), ma in scrittura produce solo una proposta di diff salvata come **Report**. L'applicazione al repository è un atto umano.

In lettura, l'Agente Docs ha un profilo diverso dagli altri: mentre Security legge pochi file mirati in profondità, Docs deve prima capire *che progetto è*, quindi parte con un'esplorazione a largo raggio (albero completo, `package.json`, README esistente). La presenza di un README trasforma il compito da generazione ad aggiornamento. In scrittura, l'output è un **Proposal** (diff verso il file di destinazione) salvato come **Report**: l'applicazione al repository resta un atto umano, coerente con RS.3.

7 Contratto delle API: comunicazione frontend–backend

Questa sezione è il riferimento operativo per chi implementa il backend (deve realizzare questi endpoint) e per il progettista frontend (deve sapere quali chiamare da ogni pagina). Elenca il contratto in modo che le due parti possano procedere in parallelo senza ambiguità.

7.1 Due piani di comunicazione distinti

Il punto da cui partire, perché è la fonte di errore più comune: esistono **due piani di comunicazione separati**, e il frontend ne usa uno solo.

Il frontend è un'applicazione React che gira **nel browser dell'utente**, non dentro la rete Docker. Raggiunge quindi il backend attraverso la porta esposta sull'host (`localhost:3000`), non attraverso l'hostname interno del container. Gli agenti Python, invece, vivono *dentro* la rete Docker e raggiungono il backend all'hostname interno (`backend:3000`) su endpoint diversi, protetti da HMAC (§ 6). Il frontend non tocca mai gli endpoint interni, e gli agenti non toccano mai quelli pubblici: è questa separazione a garantire che il token GitHub non lasci il backend (ADR-1).

Tabella 3: Indirizzi base dei servizi in ambiente di sviluppo.

Servizio	Dal browser	Nella rete Docker	
Frontend (Vite)	<code>http://localhost:5173</code>	<code>http://frontend:5173</code>	
Backend (NestJS)	<code>http://localhost:3000</code>	<code>http://backend:3000</code>	
Agenti (FastAPI)	— (non esposto)	<code>http://agents:8000</code>	

Il frontend non ha l'indirizzo del backend cablato nel codice: lo legge dalla variabile `VITE_API_BASE_URL`, così in produzione cambia senza ricompilare. Il backend adotta un prefisso globale versionato (`app.setGlobalPrefix('api/v1')`), quindi la base REST effettiva è `http://localhost:3000/api/v1`. Sia il prefisso `/api/v1` sia la scelta del canale realtime (WebSocket, § 7.3) sono proposte del progettista: se il team decide diversamente, questa sezione è il punto in cui aggiornarle.

7.2 API REST pubbliche (browser → backend)

Tutti i path che seguono sono relativi alla base `/api/v1` e documentati in Swagger, raggiungibile durante lo sviluppo a `http://localhost:3000/api/docs`. Le chiamate autenticate portano il JWT nell'header `Authorization: Bearer`.

Tabella 4: Endpoint REST pubblici, raggruppati per pagina del frontend.

Metodo e path	Scopo	Requisiti
<i>Autenticazione</i>		
POST /auth/register	Registrazione di un nuovo utente (stub nel PoC, si veda nota)	RF.1–RF.6
POST /auth/login	Login, restituisce il JWT (stub nel PoC, si veda nota)	RF.7–RF.9
GET /auth/me	Profilo dell’utente autenticato	RF.9
GET /auth/health	Healthcheck del backend (uso interno, Docker Compose)	—
<i>Credenziali di servizio</i>		
POST /credentials	Salva il token GitHub (cifrato)	RF.10–RF.11, RS.2
GET /credentials	Elenco delle credenziali configurate	RF.12
POST /credentials/:id/validate	Verifica validità e scope	RF.12, RS.4
DELETE /credentials/:id	Revoca una credenziale	RF.12
<i>Selezione del repository e dell’ambito</i>		
GET /repositories	Elenco dei repository dell’utente	RF.21–RF.23
GET /repositories/:owner/:repo/refs	Branch e tag disponibili	RF.24
GET /repositories/:owner/:repo/tree?ref=	Struttura del repository	RF.24–RF.30
POST /contexts	Crea l’AnalysisContext, ritorna contextId	RF.25–RF.31
<i>Avvio ed esecuzione</i>		
GET /operations	Operazioni disponibili (per i pulsanti)	RF.32–RF.34
POST /tasks	Avvia una o più task in batch, ritorna 202 + {taskId[], batchId}	RF.35–RF.42
GET /tasks	Elenco delle task per la dashboard	RF.43–RF.45
GET /tasks/:id	Dettaglio di una singola task	RF.45
POST /tasks/:id/cancel	Annulla una task	RF.46
<i>Storico e report</i>		
GET /reports?operation=&from=&to=	19 Storico dei report, filtrabile	RF.49–RF.51
GET /reports/:id	Dettaglio di un report	UC33

POST `/auth/register` e POST `/auth/login` sono implementati come **stub**: il primo restituisce un messaggio statico senza creare alcun utente, il secondo firma sempre lo stesso JWT per un utente fisso, senza verificare alcuna credenziale. È una scelta coerente con il perimetro dichiarato in § 3 (“il PoC assume un utente già noto”), ma va tenuta presente da chi consuma questo contratto: gli endpoint esistono e rispondono, ma non implementano ancora RF.1–RF.9.

7.3 Canale realtime (browser → backend)

L’avanzamento delle task arriva al frontend via WebSocket (Socket.IO), coerentemente con l’esecuzione asincrona su coda (ADR-3): la POST `/tasks` ritorna subito, e il frontend si iscrive al canale usando i `taskId` ricevuti. La connessione è a `ws://localhost:3000` e viene autenticata passando il JWT all’handshake. Il server emette quattro eventi verso il client; il frontend non emette eventi applicativi, solo iscrizione.

Tabella 5: Eventi WebSocket emessi dal backend verso il frontend.

Evento	Payload	Uso nel frontend
<code>task.updated</code>	<code>{ taskId, status, reportId? }</code>	Aggiorna il badge di stato in dashboard
<code>task.progress</code>	<code>{ taskId, stage, percent }</code>	Muove la barra di avanzamento
<code>task.failed</code>	<code>{ taskId, error }</code>	Mostra un toast; non blocca le altre task
<code>batch.completed</code>	<code>{ batchId, completed, failed }</code>	Abilita i controlli successivi (RF.47)

7.4 Gli schemi dei DTO principali

Per tipizzare le chiamate lato frontend bastano gli schemi degli oggetti che attraversano il confine. Gli altri (elenchi, riassunti) sono proiezioni di questi. Il contratto **Report** completo è in § 5, figura 4.

```
// --- Body di POST /contexts ---
interface CreateContextDto {
  repoOwner: string
  repoName: string
  ref: string // branch, tag o SHA
  scopeType: 'FULL_REPOSITORY' | 'FILES' | 'DIRECTORIES'
  paths?: string[] // richiesto se scopeType != FULL_REPOSITORY
  sprintId?: string // opzionale: milestone usata dall'agente Changelog
}

// --- Risposta di POST /contexts ---
interface AnalysisContextDto {
  id: string
  repoOwner: string
  repoName: string
  isPrivate: boolean
  resolvedSha: string // ref risolto a commit, per riproducibilit 
  scopeType: string
  detectedLanguages: string[]
  estimatedFileCount: number
}
```

```

}

// --- Body di POST /tasks ---
interface CreateTaskBatchDto {
  contextId: string
  operations: OperationCode[] // una o piu' operazioni, avviate
                                insieme
}

// --- Elemento restituito da GET /tasks e GET /tasks/:id ---
interface TaskDto {
  id: string
  batchId: string
  operation: OperationCode
  status: 'PENDING' | 'RUNNING' | 'COMPLETED' | 'FAILED' | 'CANCELLED'
  progressPercent: number
  currentStage: string | null
  reportId: string | null // valorizzato quando status =
                           COMPLETED
  error: { code: string, message: string, stage: string } | null
}

// --- Risposta di GET /reports/:id (envelope, vedi fig. contratto
//      Report) ---
interface ReportDto {
  id: string
  taskId: string
  operation: OperationCode
  status: 'COMPLETED' | 'FAILED'
  generatedAt: string
  summary: string
  body: Block[] // TextBlock | FindingBlock | ...
  proposal?: { targetPath: string, diffUnified: string, language: string
              }
  error?: { kind: string, message: string, stage: string }
}

type OperationCode =
  | 'DOCS_README' | 'DOCS_INLINE' | 'DOCS_API'
  | 'SECURITY_OWASP' | 'SECURITY_POLICY'
  | 'CHANGELOG_TECHNICAL' | 'CHANGELOG_BUSINESS'

```

Listing 2: Schemi dei DTO che il frontend consuma e produce. I tipi sono condivisi tra backend e frontend riusando le stesse definizioni TypeScript.

7.5 API interne (agenti → backend)

Elencate qui per completezza, ma **il frontend non le usa** e non compaiono in Swagger. Vivono sulla rete Docker a `http://backend:3000/internal/*`, autenticate via HMAC con `INTERNAL_SHARED_SECRET`. Sono la porta con cui gli agenti leggono da GitHub (§ 6): `POST /internal/github/tree`, `POST /internal/github/file`, `POST /internal/github/issues` (i tre tool di lettura) e `POST /internal/tasks/:id/progress` (il callback di avanzamento, che il backend inoltra sul WebSocket).

8 Orchestrazione ed esecuzione

8.1 Instradamento deterministico

Anche nella sua forma minima, l'instradamento operazione→agente è **deterministico**, come impone il vincolo RV.1: non c'è alcun LLM nella logica di smistamento. Il componente responsabile (**AgentRegistry**) è una mappa statica popolata a bootstrap; dato un **OperationCode**, la destinazione è determinata senza inferenza. In sede di revisione è la prima classe da mostrare, perché il vincolo si verifica a occhio.

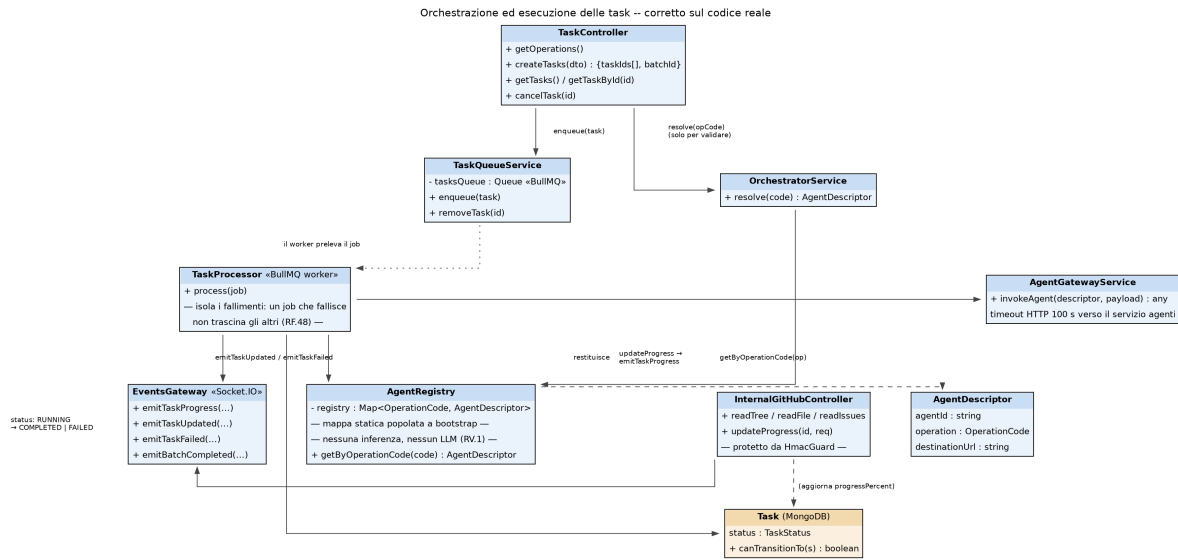


Figura 7: Orchestrazione ed esecuzione. **AgentRegistry** è il cuore di RV.1 (mappa statica, nessun LLM). Il **TaskProcessor** isola i fallimenti: un job che fallisce non trascina gli altri (RF.48).

8.2 Ciclo di vita di una Task

Una Task attraversa cinque stati (figura 8): le transizioni ammesse sono solo quelle disegnate, e **Task.canTransitionTo()** rifiuta le altre. Nessuno stato terminale ha uscite: un rilancio è una Task nuova che tiene il riferimento alla precedente, così lo storico dei report resta interpretabile.

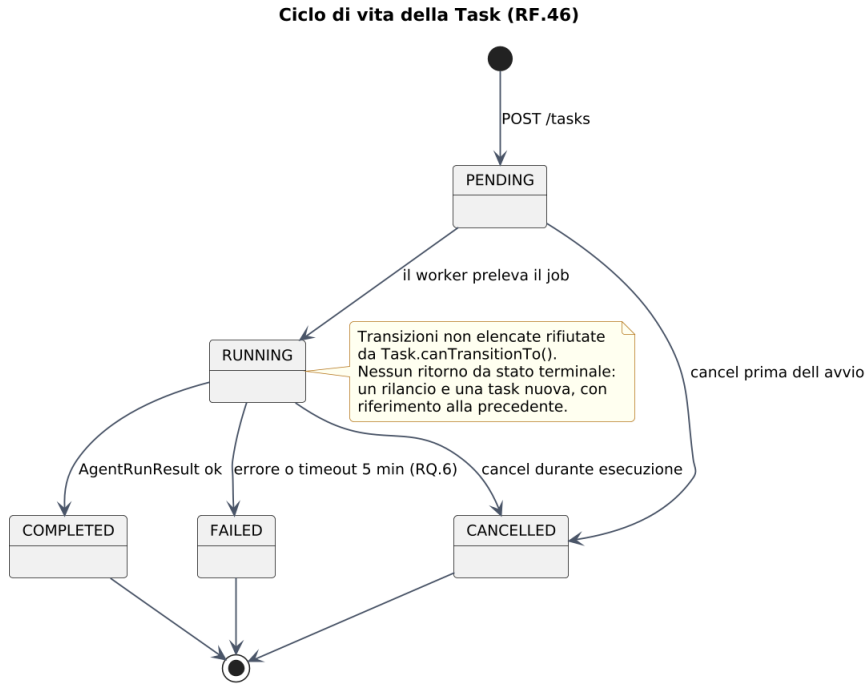


Figura 8: Ciclo di vita della Task (RF.46). Il superamento del timeout configurato per l’agente (90 s per Docs e Changelog, 180 s per Security, che richiede più cicli di tool-calling) porta a **FAILED** restando dentro il contratto **Report**.

8.3 Il giro completo end-to-end

La figura 9 mostra lo scambio di messaggi per un’operazione concreta (scansione OWASP su un repository privato). Il blocco centrale è quello su cui fermarsi in revisione: l’agente chiede l’albero, *decide* di aprire un file, lo chiede, lo riceve. Nessuno di questi passi è programmato da noi: sono tool call scelte dal modello, e ognuna passa dal backend, che tiene il token e registra l’accesso. Un solo giro attraversa React, Vite, TanStack, NestJS, TypeScript, MongoDB, Redis, Python, LangGraph, Octokit, l’API GitHub, Docker, WebSocket e Swagger: è la dimostrazione di integrazione che le stesure precedenti non fornivano. Questo pattern di tool-calling guidato dal modello, qui illustrato sulla scansione OWASP, è specifico dell’agente Security: Docs e Changelog non lo attivano, perché il loro **ContextLoader** carica l’intero contesto necessario in un unico passo deterministico prima di interpellare il modello (§ 4.2).

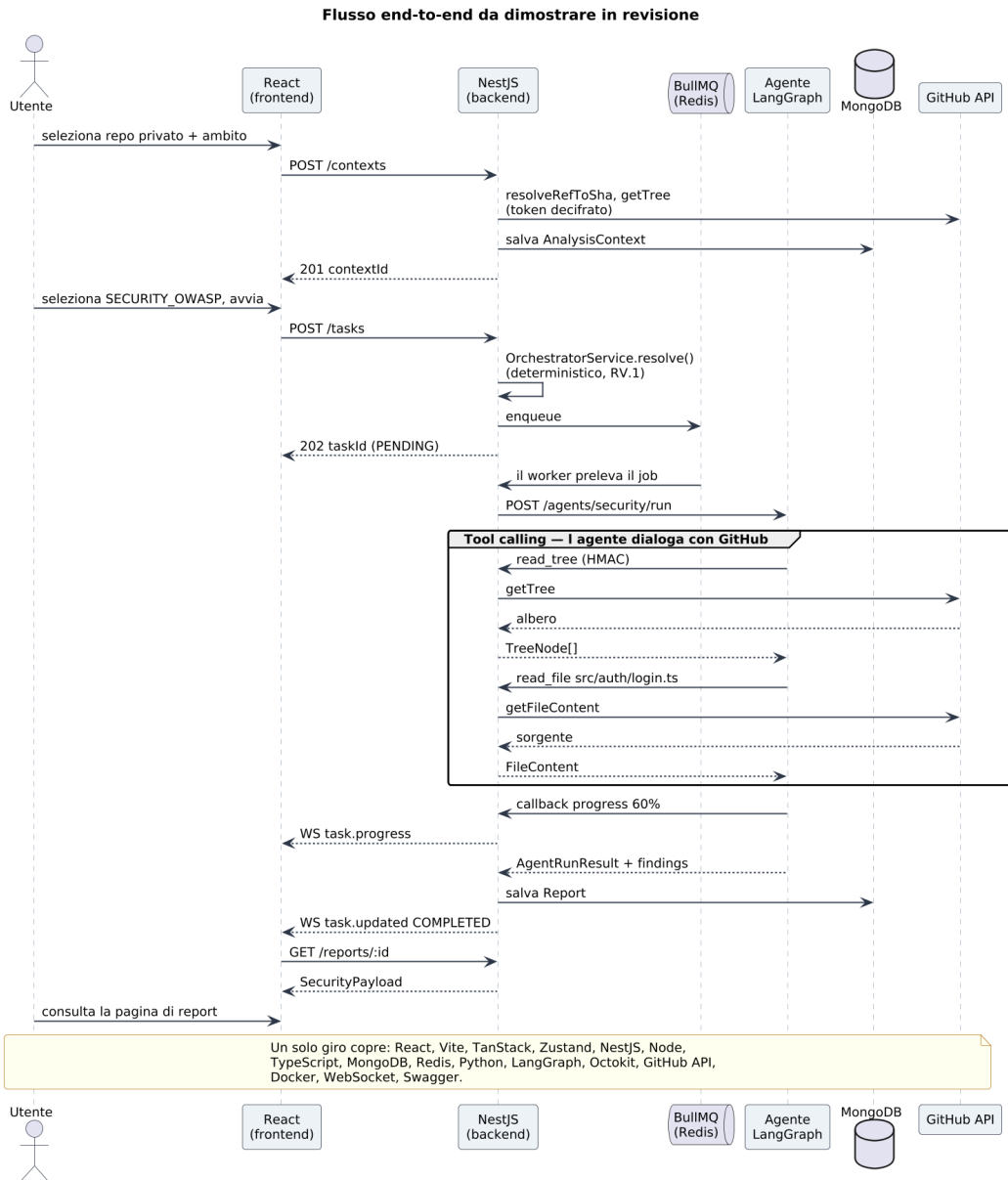


Figura 9: Sequenza end-to-end di una scansione. Il tool-calling dell’agente (blocco centrale) attraversa la facade GitHub, non GitHub direttamente.

9 Cosa implemento per ciascun agente

Poiché grafo, provider, envelope e gestione degli errori sono condivisi, “implementare un agente” significa scrivere sempre e solo tre cose: l’adattatore che carica il contesto (dai tool GitHub), il profilo con il suo template di prompt, e il parser che rilegge la risposta. L’esposizione via backend e la visualizzazione via frontend sono uguali per tutti e tre, scritte una sola volta contro il contratto **Report**. La figura 10 mostra la gerarchia.

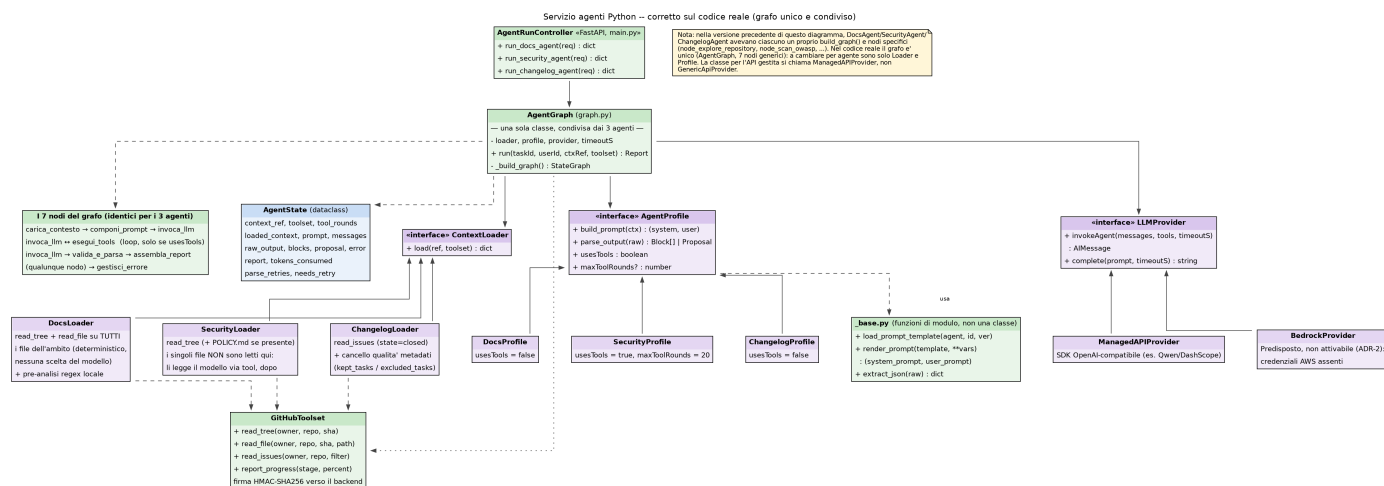


Figura 10: Il servizio agenti. Il grafo LangGraph è unico e condiviso da tutti gli agenti (classe `AgentGraph`, sette nodi generici, cfr. § 4.2): non esiste un `build_graph()` né un insieme di nodi distinto per sottoclasse. Ciò che varia per agente sono solo il `Loader` e il `Profile` iniettati (le porte descritte in § 4.2). `PromptRepository` rende misurabile l'isolamento dei prompt (RQ.4); `GitHubToolset` espone i tool di lettura.

9.1 Agente Docs — documentazione inline

L'adattatore di contesto legge il codice dal repository **tramite i tool della facade** (`read_tree`, `read_file`), limitandosi all'ambito selezionato e ai soli linguaggi supportati. Prima di chiamare il modello c'è una piccola preparazione: attraversare quei file e individuare le unità che meritano documentazione (funzioni, classi, metodi) che ne sono prive o il cui commento è disallineato. Per un PoC non serve un analizzatore sintattico completo: basta riconoscere le definizioni senza un blocco di documentazione sopra. Il profilo carica il proprio template e vi inietta il codice di quelle unità, chiedendo al modello il commento nella convenzione corretta (JSDoc per TS/JS, docstring per Python). Il parser verifica che ogni commento sia ben formato e lo riaggancia a file e riga; il report è una `Proposal` di diff, mostrata nel frontend con una vista diff dedicata. Dove il codice è troppo intricato, l'agente emette un avviso strutturato invece di forzare un commento sbagliato.

Implemento: lettura del codice dall'ambito via facade, rilevamento delle unità non documentate, generazione del commento conforme, diff proposto, avviso per complessità eccessiva, visualizzazione del diff nell'interfaccia.

Non implemento: README di progetto e documentazione API (le altre due operazioni di Docs), e l'apertura effettiva della Pull Request.

9.2 Agente Security — scansione vulnerabilità OWASP

L'adattatore di contesto usa gli stessi tool di lettura; nella variante policy-as-code legge in più il file `POLICY.md` alla radice. Qui non c'è pre-analisi: il codice dell'ambito, spezzato in porzioni che stanno nella finestra di contesto, va direttamente al modello. Il profilo carica il template di scansione, che istruisce il modello a cercare i pattern dell'OWASP Top 10 (SQL injection, cross-site scripting, segreti in chiaro, crittografia insicura, controllo accessi incompleto) e a restituire per ogni riscontro categoria, gravità, file e righe, spiegazione e rimedio. Il parser attende una risposta strutturata, la valida, normalizza la gravità sui livelli previsti e controlla i riferimenti alle righe; se la risposta è malformata scatta il ramo di parsing, se il modello è troppo lento quello di timeout, fissato per questo agente a 180 secondi (più ampio del default degli altri due, perché la scansione richiede più cicli di lettura file). Il report è una lista di `FindingBlock`, ciascuno con il suo rimedio, mostrata nel frontend come elenco filtrabile per gravità.

Implemento: estrazione del codice via facade, scansione OWASP Top 10, output strutturato (categoria/gravità/posizione/rimedio), gestione di timeout e parsing, elenco dei finding nell'interfaccia. In opzione, la variante policy-as-code su `POLICY.md`, che riusa identico il grafo cambiando solo il template.

Non implemento: copertura esaustiva dell'intera OWASP Top 10 e applicazione automatica dei rimedi — l'agente segnala, è il Security Auditor a validare.

9.3 Agente Changelog — changelog tecnico

Rispetto alle stesure precedenti, l'adattatore di contesto **non** legge più una fixture JSON: interroga **GitHub Issues dal vivo** tramite il tool `read_issues`, recuperando le storie di uno Sprint (titolo, descrizione, criteri di accettazione, stato, etichette, milestone). Il lavoro prima del modello è interno: filtrare le issue effettivamente chiuse per lo Sprint indicato e poi passarle al controllo di qualità dei metadati, che mette da parte quelle con descrizione vuota o troppo povera. Si segue il percorso “procedi escludendo”: si registra quali task sono state scartate e si prosegue solo con quelle valide, così il report è onesto su cosa copre. Il profilo carica il template del changelog tecnico e chiede al modello un resoconto per sviluppatori in Markdown; siccome l'output è prosa, la validazione è più lieve, e il testo viene arricchito con i collegamenti ai ticket. Il report è composto da un unico `TextBlock` in Markdown, generato dal modello, che contiene già tutte le voci incluse con i link ai ticket; gli eventuali `ChangelogItemBlock` che seguono non sono altre voci del changelog, ma l'elenco — per trasparenza — delle issue *escluse* dal cancello di qualità sui metadati. Il tutto è mostrato nel frontend come Markdown renderizzato, con la lista delle esclusioni in coda.

Implemento: lettura delle issue da GitHub dal vivo, filtro delle chiuse, cancello di qualità sui metadati (scarto delle povere), sintesi tecnica in Markdown, arricchimento con i link ai ticket, rendering Markdown nell'interfaccia.

Non implemento: il changelog di business (sarebbe un nodo in più che rigira il testo tecnico in un secondo prompt).

9.4 Il filo comune

In tutti e tre i casi l'unica cosa che scrivo davvero di nuovo, lato agente, è “da dove leggo” (quale tool della facade) e “cosa chiedo e come rileggo la risposta”. Orchestrazione dei nodi, chiamata al modello con timeout, forma del report e gestione degli errori arrivano dallo scheletro condiviso. A questo si aggiunge il filo comune lato stack applicativo: gli endpoint REST, la persistenza su MongoDB e i componenti dell'interfaccia che mostrano il **Report** sono scritti una sola volta e riusati identici, perché tutti parlano lo stesso contratto.

10 Decisioni architetturali (ADR)

Queste decisioni sono vincolanti per l'implementazione: sono i punti dove la strada più comoda viola un requisito. Gli sviluppatori le seguono anche quando una scorciatoia sembra allettante.

ADR-1 — Il token GitHub non lascia NestJS. Gli agenti devono dialogare con GitHub, ma RS.3 vieta loro la scrittura e RS.2 impone token cifrati. **Decisione:** facade read-only su NestJS, esposta agli agenti come tool `LangGraph`. Il token resta cifrato nel backend; l'agente decide cosa leggere e iterare, ma ogni chiamata passa dalla whitelist. **Da non fare:** passare il token al servizio Python perché “è più semplice”. Sposterebbe il segreto oltre un secondo confine e renderebbe RS.3 da verificare in due codebase.

ADR-2 — L’LLM dietro una porta, fornitore della famiglia di Bedrock. Credenziali AWS assenti. **Decisione:** LLMProvider con implementazione attiva verso API gestita della stessa famiglia di modelli che useremo su Bedrock, e BedrockProvider scritto ma non attivo. **Da non fare:** eseguire un modello locale, che non direbbe nulla di utile sul prodotto e costringerebbe a ritardare i prompt.

ADR-3 — Coda per l’esecuzione, non chiamata sincrona. RQ.6 fissa un obiettivo di tempo di risposta contenuto per agente, e i fallimenti vanno isolati (RF.48). Nel PoC il timeout è configurato a 90 secondi per Docs e Changelog e a 180 secondi per Security (che itera più cicli di lettura file); il divario rispetto all’obiettivo RQ.6 andrà richiuso nelle fasi successive, ottimizzando prompt e numero di letture. **Decisione:** POST /tasks accoda e ritorna subito; un worker esegue; gli aggiornamenti arrivano via WebSocket. **Da non fare:** eseguire l’agente dentro il ciclo della richiesta HTTP, che accoppierebbe l’interfaccia ai tempi del modello.

ADR-4 — Gli artefatti degli agenti sono proposte, non modifiche. **Decisione:** l’output è un Report immutabile scaricabile, mai un commit. Il token richiesto ha scope di sola lettura. **Evoluzione futura:** un’eventuale apertura di Pull Request nel prodotto dovrà essere innescata esplicitamente da un umano dopo aver visto il diff, e usare un token con scope di scrittura distinto. Questo va deciso prima di disegnare la UI del report, perché ne cambia i pulsanti.

11 Come validiamo il PoC

Il PoC si aggancia direttamente alle metriche del Piano di Qualifica.

L’**accuratezza** (obiettivo > 85%) si misura preparando un piccolo golden set di file con vulnerabilità note e confrontando l’output dell’agente Security. Con il provider via API gestita, questa metrica va verificata presto sul fornitore effettivamente scelto.

Il **tempo di risposta** si misura per singola esecuzione, dall’invocazione dell’endpoint REST alla disponibilità del report, e include l’overhead di persistenza su MongoDB e delle letture GitHub (mitigate dalla cache). L’obiettivo di RQ.6 resta un tempo di risposta contenuto; i timeout attualmente configurati nel PoC (90 s per Docs e Changelog, 180 s per Security) sono più ampi del target e vanno visti come un tetto di sicurezza da stringere, non come il valore a cui puntare.

L’**isolamento dei prompt** è verificabile per costruzione, perché i prompt sono file esterni. Conviene aggiungere un test statico che fallisce se un modulo agente contiene literal di stringa oltre una soglia: rende la metrica MPD.14 una prova automatica.

La **copertura dei test** è raggiungibile perché sia LLMProvider sia la facade GitHub sono dietro interfaccia: nei test si iniettano un provider simulato e un client GitHub finto con risposte registrate, così i test girano offline, deterministici e a costo zero.

Si aggiunge la verifica dell’**integrazione end-to-end**: un test che, partendo da una chiamata simulata dal frontend, verifica che il backend instradi correttamente l’agente, che l’agente legga via facade (con client GitHub finto), che il report sia persistito su MongoDB e restituito nel formato atteso.

Uso dei dati per l’addestramento (RS.5). Il vincolo RS.5 impone di garantire che il codice sorgente inviato al modello **non venga trattenuto né usato per addestrare modelli di terzi**. Questo punto merita attenzione perché, con la scelta di un’API gestita, la garanzia dipende dal fornitore e dal canale, e i due canali che il progetto usa in momenti diversi **non offrono la stessa copertura in modo automatico**.

Quando il modello sarà servito da **AWS Bedrock**, in produzione, la copertura è netta: AWS dichiara che i dati dei clienti non vengono mai usati per addestrare i modelli sottostanti, e

questo vale anche per i modelli open-weight di terzi serviti tramite Bedrock, perché l'inferenza avviene nel perimetro AWS. In quella configurazione RS.5 è soddisfatto dai termini del servizio.

Durante il **PoC**, però, il modello non passa da Bedrock ma dall'**API diretta del fornitore** (nel nostro esempio, DashScope di Alibaba Cloud): la garanzia di Bedrock non si estende a questo canale, che è retto dai propri termini di servizio. La documentazione dei fornitori è esplicita nel rimandare le politiche sui dati al provider effettivamente usato. Ne consegue un adempimento preciso, non tecnico ma con scadenza esterna, da svolgere **prima** di inviare al modello il primo file di un repository privato del proponente:

1. verificare, nei termini del piano API scelto (per Qwen: il piano DashScope internazionale, regione di Singapore/Model Studio), la clausola specifica sulla ritenzione dei dati e sul loro eventuale uso per addestramento, controllando se esista un'opzione o un tier che escluda esplicitamente tale uso;
2. mettere a verbale l'esito della verifica, con data e versione dei termini consultati, perché è la forma in cui RS.5 è dimostrabile in sede di collaudo;
3. tracciare il requisito nel Piano di Qualifica come **controllo documentale**, non come test automatico: nessun test può dimostrare che un terzo non usi i dati, solo i termini contrattuali possono.

Se la verifica sui termini dell'API diretta non desse garanzie sufficienti, l'alternativa a costo maggiore è anticipare l'uso di un canale con garanzie enterprise sui dati (Bedrock stesso, se e quando le credenziali arrivano, o un intermediario equivalente), accettando però che questo tolga al PoC la sua indipendenza dalle credenziali AWS. È una decisione da prendere consapevolmente, non un dettaglio da rimandare.

12 Sintesi e prossimi passi

Il PoC dimostra molto più di “un agente funziona”: prova che i tre agenti condividono backbone, provider e contratto di output, differendo solo in due punti localizzati, e prova che l'intero stack applicativo (backend NestJS/MongoDB, frontend React, servizio agenti Python) si integra con quel core e con GitHub in modo end-to-end. È un'affermazione architetturale più forte di quella delle stesure precedenti, e copre la soglia di integrazione richiesta dal proponente. Resta un PoC: una operazione per agente, orchestrazione minima, gli stessi rami d'errore per tutti, nessuna scrittura verso l'esterno e infrastruttura locale in attesa della migrazione ad AWS.

I passi immediatamente successivi, una volta condiviso questo perimetro, sono: la stesura dei template di prompt (Docs, OWASP, Changelog) che i profili caricheranno; la scelta e la validazione del fornitore LLM sul golden set; l'implementazione della facade GitHub con la sua whitelist e i tre tool; e la preparazione dei componenti di interfaccia condivisi per la visualizzazione del Report.